

Mutex

Mutex

- 1. Mutex Overview
 - 1.1 What is a Mutex
 - 1.2 Importance and Application Scenarios
 - 1.3 This Project Demonstrates
- 2. Core Concepts
 - 2.1 How Mutex Works
 - 2.2 Mutex API
 - 2.3 Mutex vs Binary Semaphore
 - 2.4 Shared Resource Protection Principles
- 3. Project Architecture and Flow
- 4. Hardware Dependencies
- 5. Source Code Analysis
 - 5.1 Header Files and Macro Configuration
 - 5.2 key_init() — Button GPIO Initialization
 - 5.3 rgb_init() — WS2812 LED Initialization
 - 5.4 rgb_set() / rgb_rainbow_loop() / rgb_red_loop() — LED Display Functions
 - 5.5 check_winner() — Win Condition Check
 - 5.6 vProducer() — Producer Task
 - 5.7 vConsumer1_Human() — Human Consumer
 - 5.8 vConsumer2_AI() — AI Consumer
 - 5.9 app_main() — Main Entry
- 6. Operation Flow
 - 6.1 Compilation and Flashing Flow
 - 6.2 Normal Operation
 - 6.3 Edge Cases
- 7. FAQ

1. Mutex Overview

1.1 What is a Mutex

Mutex is FreeRTOS's core mechanism for protecting **shared resources**. When multiple tasks need to access the same global variable, peripheral, or data structure, mutex ensures that **only one task** can enter the critical section at any given time, preventing data races.

1.2 Importance and Application Scenarios

Mutex is one of the **most commonly used, most fundamental** synchronization mechanisms in RTOS multi-tasking programming, and almost any scenario involving multi-task shared resources relies on it:

Application Scenario	Description
Shared memory protection	When multiple tasks simultaneously read/write global variables/structs/arrays, must lock to ensure data consistency

Application Scenario	Description
Peripheral exclusive access	When multiple tasks operate on the same SPI/I2C/UART bus, prevent bus conflicts and data corruption
Queue/linked list operations	Insert/delete operations on FreeRTOS queues or custom linked lists require atomic protection
Sensor data acquisition	Acquisition task writes, display task reads, must be mutually exclusive to prevent reading "half-new half-old" data
File system writing	Multiple tasks writing to the same file causes data corruption, mutex ensures serialized writes
Display screen refresh	When multiple tasks update different areas of the same screen, lock is needed to protect frame buffer

Core Value: Mutex is not only a tool for protecting data, but also the cornerstone of ensuring system **stability and predictability**. Unlocked shared resource access is one of the **hardest to reproduce, hardest to debug** bugs in embedded systems.

1.3 This Project Demonstrates

This project demonstrates mutex through a **"Grab Goods Game" (Human vs AI)**: Producer produces goods every second, human holds BOOT button to grab lock→releases to grab goods, AI automatically grabs lock→grabs goods→cools down for 1s. First to grab 10 goods wins. Human wins → rainbow LED cycling, AI wins → solid red LED.

Through this game, you will intuitively understand: **lock exclusivity** (human keeps holding → AI can never get lock), **critical section protection** (all stock/score operations are inside the lock), and **deadlock prevention** (each task strictly follows Take → Operation → Give order).

2. Core Concepts

2.1 How Mutex Works



| Critical |

| Section |

2.2 Mutex API

```
// Create mutex
SemaphoreHandle_t xSemaphoreCreateMutex(void);

// Take lock (enter critical section)
xSemaphoreTake(xMutex, portMAX_DELAY);

// Give lock (exit critical section)
xSemaphoreGive(xMutex);
```

2.3 Mutex vs Binary Semaphore

Feature	Mutex	Binary Semaphore
Purpose	Mutual exclusion — protect shared resources	Synchronization — event notification
Priority inheritance	✓ (auto elevate holder priority)	X
Recursive acquisition	Configurable recursive mutex available	Not available
Releaser	Must be acquirer (same task Take → Give)	Any task can Give
Use in ISR	X (cannot Take/Give in ISR)	✓ (GiveFromISR)
Initial value	1 (available, not held by any task)	Usually 0

Mutex priority inheritance mechanism is key to solving priority inversion. When a high-priority task waits for a lock, the low-priority task holding the lock is temporarily elevated to high priority to ensure quick lock release.

2.4 Shared Resource Protection Principles

```
All code accessing g_goods must:
1. xSemaphoreTake(xMutex, ...) ← Take lock
2. Read/modify g_goods          ← Critical section
3. xSemaphoreGive(xMutex)       ← Give lock

Missing any step = data race = bug
```

3. Project Architecture and Flow

```
app_main()
├─ rgb_init() + key_init()           ← WS2812 (GPIO48) + Button (GPIO0)
├─ xSemaphoreCreateMutex()          ← Create mutex (protects: g_goods, g_human_score,
g_ai_score)
├─ xTaskCreate(vProducer)           ← Producer (Prio 2)
├─ xTaskCreate(vConsumer1_Human)    ← Human (Prio 3)
├─ xTaskCreate(vConsumer2_AI)      ← AI (Prio 2)

Producer —(take→produce n→give)→ g_goods    ← Shared resource
Human    —(hold→take...release→grab→give)→ g_human_score
AI       —(take→grab→give→cool 1s)→ g_ai_score

Human holds button continuously → Lock always held by Human → AI blocks permanently at
xSemaphoreTake

                                → Demonstrates mutex exclusivity
```

Game Rules:

Parameter	Value
Win condition	First to grab 10 goods
Producer	Produces 2~5 randomly per second
AI cooldown	Waits 1 second after successful grab
Human operation	Hold BOOT = hold lock, release = grab + give lock
Human wins → Rainbow cycling	AI wins → Solid red

4. Hardware Dependencies

Pin	Function	Description
GPIO 0	BOOT button	Human uses button to grab lock and goods
GPIO 48	WS2812 RGB LED	Human wins rainbow / AI wins red

Required Components: `freertos/FreeRTOS`, `freertos/task`, `freertos/semphr`, `driver/gpio`, `led_strip`, `esp_log`, `esp_random`

This project uses a **third-party library** again: For details, review (**Third-party library installation process: 2.Basic Peripherals → 10.RGB LED Effects**)

Note: `led_strip` is a third-party component provided by ESP-IDF Component Registry, not built into IDF. Before use, declare dependency in `main/idf_component.yml`:

```
## IDF Component Manager Manifest File
dependencies:
idf:
  version: '>=4.1.0'
  espressif/led_strip: ^3.0.3
```

After declaration, execute `idf.py build`, and component manager will automatically download component to `managed_components/` directory.

5. Source Code Analysis

Full source code: [Source Code](#) → [ESP-IDF](#) → [4.FreRTOS](#) → [2. RTOS_Inter-task_communication](#) → [RTOS-MUTEX](#)

5.1 Header Files and Macro Configuration

This section defines GPIO pins, game parameters, and all shared resource variables protected by mutex.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_log.h"
#include "esp_random.h"
#include "driver/gpio.h"
#include "led_strip.h"

static const char *TAG = "MUTEX_GAME";

// ---- GPIO Pin Definitions ----
#define KEY_GPIO      0
#define BLINK_GPIO    48
#define LED_NUM       1
#define DEBOUNCE_MS   50

// ---- Game Parameters ----
#define WIN_COUNT      10
#define PRODUCE_MIN    2
#define PRODUCE_MAX    5
#define AI_COOLDOWN_MS 1000

// ---- Rainbow Colors (WS2812 GRB Order) ----
typedef struct {
    uint8_t green;
    uint8_t red;
    uint8_t blue;
} led_color_t;

static const led_color_t rainbow_colors[] = {
    {0, 150, 0 }, // Red
    {60, 150, 0 }, // Orange
```

```

    {150, 150, 0 },    // Yellow
    {150, 0,  0 },    // Green
    {150, 0,  150},    // Cyan
    {0,  0,  150},    // Blue
    {0,  120, 150},    // Purple
};
#define COLOR_COUNT 7

// ---- Shared Resources (all protected by mutex) ----
static SemaphoreHandle_t xMutex = NULL;
static int g_goods = 0;
static int g_human_score = 0;
static int g_ai_score = 0;
static bool g_game_over = false;
static int g_winner = 0;

static led_strip_handle_t led_strip;

```

5.2 key_init() — Button GPIO Initialization

Configure BOOT button (GPIO0) as internal pull-up input mode, low level when pressed.

```

static void key_init(void)
{
    gpio_config_t cfg = {
        .pin_bit_mask = 1ULL << KEY_GPIO,
        .mode          = GPIO_MODE_INPUT,
        .pull_up_en     = GPIO_PULLUP_ENABLE,
        .pull_down_en   = GPIO_PULLDOWN_DISABLE,
        .intr_type      = GPIO_INTR_DISABLE,
    };
    gpio_config(&cfg);
}

```

5.3 rgb_init() — WS2812 LED Initialization

Initialize WS2812 strip on GPIO48 via RMT peripheral.

```

static void rgb_init(void)
{
    led_strip_config_t strip_cfg = {
        .strip_gpio_num    = BLINK_GPIO,
        .max_leds          = LED_NUM,
        .led_model         = LED_MODEL_WS2812,
        .color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB,
        .flags.invert_out   = false,
    };
    led_strip_rmt_config_t rmt_cfg = {
        .resolution_hz = 10 * 1000 * 1000,
        .flags.with_dma = false,
    };
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_cfg, &rmt_cfg, &led_strip));
}

```

```
    led_strip_clear(led_strip);  
}
```

5.4 rgb_set() / rgb_rainbow_loop() / rgb_red_loop() — LED Display Functions

Three LED control functions: set solid color, rainbow cycle, solid red.

```
static void rgb_set(uint8_t r, uint8_t g, uint8_t b)  
{  
    led_strip_set_pixel(led_strip, 0, r, g, b);  
    led_strip_refresh(led_strip);  
}  
  
static void rgb_rainbow_loop(void)  
{  
    while (1) {  
        for (int i = 0; i < COLOR_COUNT; i++) {  
            rgb_set(rainbow_colors[i].red, rainbow_colors[i].green, rainbow_colors[i].blue);  
            vTaskDelay(pdMS_TO_TICKS(500));  
        }  
    }  
}  
  
static void rgb_red_loop(void)  
{  
    while (1) {  
        rgb_set(150, 0, 0);  
        vTaskDelay(pdMS_TO_TICKS(500));  
    }  
}
```

5.5 check_winner() — Win Condition Check

Check if either side's score has reached win condition, **must be called while holding mutex** (both score and flag are shared resources).

```
static void check_winner(void)  
{  
    if (g_human_score >= WIN_COUNT) {  
        g_game_over = true;  
        g_winner = 1;  
        ESP_LOGI(TAG, "===== HUMAN WINS! =====");  
    } else if (g_ai_score >= WIN_COUNT) {  
        g_game_over = true;  
        g_winner = 2;  
        ESP_LOGI(TAG, "===== AI WINS! =====");  
    }  
}
```

5.6 vProducer() — Producer Task

Producer produces 2~5 goods randomly per second, following the standard mutex flow: **Take lock** → **Critical section operation** → **Give lock**.

```
static void vProducer(void *pvParameters)
{
    while (!g_game_over) {
        int n = PRODUCE_MIN + (esp_random() % (PRODUCE_MAX - PRODUCE_MIN + 1));

        xSemaphoreTake(xMutex, portMAX_DELAY);
        if (!g_game_over) {
            g_goods += n;
            ESP_LOGI(TAG, "Producer +%d → goods=%d | Human=%d AI=%d",
                     n, g_goods, g_human_score, g_ai_score);
        }
        xSemaphoreGive(xMutex);

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
    ESP_LOGI(TAG, "Producer exit");
    vTaskDelete(NULL);
}
```

5.7 vConsumer1_Human() — Human Consumer

Button-controlled goods grabbing, **core design: keeping hold = lock always held**, intuitively demonstrating mutex exclusivity.

```
static void vConsumer1_Human(void *pvParameters)
{
    while (!g_game_over) {
        if (gpio_get_level(KEY_GPIO) == 0) {
            vTaskDelay(pdMS_TO_TICKS(DEBOUNCE_MS));
            if (gpio_get_level(KEY_GPIO) == 0) {

                ESP_LOGI(TAG, ">>> Human PRESS → taking lock...");
                xSemaphoreTake(xMutex, portMAX_DELAY);
                ESP_LOGI(TAG, ">>> Human GOT LOCK (AI blocked now!)");

                while (gpio_get_level(KEY_GPIO) == 0) {
                    vTaskDelay(pdMS_TO_TICKS(20));
                }

                if (g_goods > 0 && !g_game_over) {
                    g_goods--;
                    g_human_score++;
                    ESP_LOGI(TAG, "<<< Human RELEASE → grabbed! goods=%d Human=%d AI=%d",
                             g_goods, g_human_score, g_ai_score);
                    check_winner();
                } else if (!g_game_over) {
                    ESP_LOGI(TAG, "<<< Human RELEASE → no goods, unlock anyway");
                }
            }
        }
    }
}
```



```

    }

    xSemaphoreGive(xMutex);
    ESP_LOGI(TAG, "<<< Human UNLOCKED (AI can compete now)");
}
}
vTaskDelay(pdMS_TO_TICKS(10));
}
ESP_LOGI(TAG, "Consumer1 (Human) exit");
vTaskDelete(NULL);
}

```

Key Design: During human holding button, lock is continuously held. AI's call to `xSemaphoreTake` blocks forever. Only at moment of release does it give lock and grab goods. This truly reflects the mutual exclusion semantics of "holding resource without release".

5.8 vConsumer2_AI() — AI Consumer

AI automatically grabs lock → grabs goods → gives lock → cools down for 1 second. When human holds button, AI blocks permanently at `xSemaphoreTake()`.

```

static void vConsumer2_AI(void *pvParameters)
{
    while (!g_game_over) {
        ESP_LOGI(TAG, "--- AI trying to take lock...");

        xSemaphoreTake(xMutex, portMAX_DELAY);
        ESP_LOGI(TAG, "--- AI GOT LOCK");

        if (g_goods > 0 && !g_game_over) {
            g_goods--;
            g_ai_score++;
            ESP_LOGI(TAG, "--- AI grabbed! goods=%d Human=%d AI=%d",
                    g_goods, g_human_score, g_ai_score);
            check_winner();
        } else if (!g_game_over) {
            ESP_LOGI(TAG, "--- AI got lock but no goods, release");
        }

        xSemaphoreGive(xMutex);
        ESP_LOGI(TAG, "--- AI UNLOCKED");

        vTaskDelay(pdMS_TO_TICKS(AI_COOLDOWN_MS));
    }
    ESP_LOGI(TAG, "Consumer2 (AI) exit");
    vTaskDelete(NULL);
}

```

AI's 1-second cooldown limits its speed, giving human operating space. Without cooldown, AI might grab all 10 before human can react.

5.9 app_main() — Main Entry

Main function initializes hardware and mutex, creates three tasks, then waits for game end, driving different LED effects based on winner.

```
void app_main(void)
{
    rgb_init();
    key_init();
    rgb_set(0, 0, 0);

    xMutex = xSemaphoreCreateMutex();
    if (xMutex == NULL) {
        ESP_LOGE(TAG, "Failed to create mutex");
        return;
    }

    ESP_LOGI(TAG, "=== GAME START === First to %d wins!", WIN_COUNT);
    ESP_LOGI(TAG, "Human: press & hold BOOT key → lock, release → grab + unlock");
    ESP_LOGI(TAG, "AI: try lock → grab → unlock → 1s cooldown");

    xTaskCreate(vProducer, "Producer", 2048, NULL, 2, NULL);
    xTaskCreate(vConsumer1_Human, "Human", 2048, NULL, 3, NULL);
    xTaskCreate(vConsumer2_AI, "AI", 2048, NULL, 2, NULL);

    while (!g_game_over) {
        vTaskDelay(pdMS_TO_TICKS(200));
    }

    if (g_winner == 1) {
        ESP_LOGI(TAG, "Human wins → Rainbow LED cycling");
        rgb_rainbow_loop();
    } else {
        ESP_LOGI(TAG, "AI wins → Solid Red LED");
        rgb_red_loop();
    }
}
```

6. Operation Flow

6.1 Compilation and Flashing Flow

For detailed flashing process: 1. Before Development → 3. Build and Flash

6.2 Normal Operation

1. After flashing, power on → Game starts

```
I (282) MUTEX_GAME: === GAME START === First to 10 wins!
I (282) MUTEX_GAME: Human: press & hold BOOT key → lock, release → grab + unlock
```

2. Producer produces 2~5 goods per second, serial port prints stock in real time

```
I (3292) MUTEX_GAME: Producer +5 → goods=9 | Human=0 AI=3
```

3. **Hold BOOT button continuously** → Serial prints `Human GOT LOCK (AI blocked now!)`, AI blocks at `xSemaphoreTake`

```
I (762) MUTEX_GAME: >>> Human PRESS → taking lock...  
I (762) MUTEX_GAME: >>> Human GOT LOCK (AI blocked now!)  
I (1312) MUTEX_GAME: --- AI trying to take lock...
```

4. **Release BOOT button** → Grabbed one goods, print score

```
I (4642) MUTEX_GAME: <<< Human RELEASE → grabbed! goods=1 Human=1 AI=1
```

5. AI automatically grabs goods (cools down for 1 second after each grab)

```
I (302) MUTEX_GAME: --- AI GOT LOCK  
I (302) MUTEX_GAME: --- AI grabbed! goods=4 Human=0 AI=1  
I (312) MUTEX_GAME: --- AI UNLOCKED  
I (1292) MUTEX_GAME: Producer +5 → goods=9 | Human=0 AI=1  
I (1312) MUTEX_GAME: --- AI trying to take lock...  
I (1312) MUTEX_GAME: --- AI GOT LOCK  
I (1312) MUTEX_GAME: --- AI grabbed! goods=8 Human=0 AI=2  
I (1312) MUTEX_GAME: --- AI UNLOCKED  
I (2292) MUTEX_GAME: Producer +5 → goods=13 | Human=0 AI=2
```

6. Someone reaches 10 points → Game over → LED shows result

```
I (3912) MUTEX_GAME: <<< Human RELEASE → grabbed! goods=0 Human=10 AI=4  
I (3912) MUTEX_GAME: ===== HUMAN WINS! =====  
I (3912) MUTEX_GAME: <<< Human UNLOCKED (AI can compete now)  
I (3912) MUTEX_GAME: Human wins → Rainbow LED cycling  
I (3922) MUTEX_GAME: Consumer1 (Human) exit  
I (4442) MUTEX_GAME: Producer exit  
I (4452) MUTEX_GAME: Consumer2 (AI) exit
```

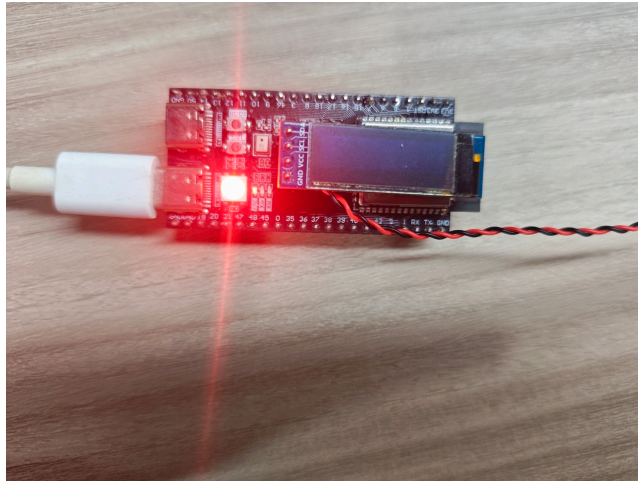
```
I (9312) MUTEX_GAME: ===== AI WINS! =====  
I (9312) MUTEX_GAME: --- AI UNLOCKED  
I (9312) MUTEX_GAME: AI wins → Solid Red LED  
I (9322) MUTEX_GAME: Consumer1 (Human) exit  
I (10292) MUTEX_GAME: Producer exit  
I (10312) MUTEX_GAME: Consumer2 (AI) exit
```

Log example:

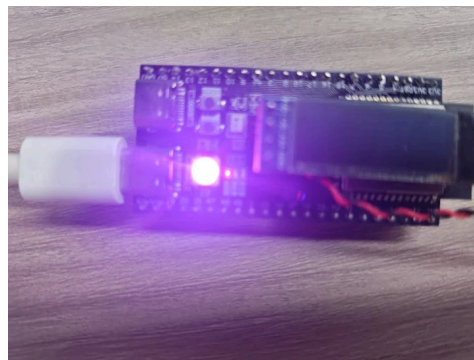
```
I (1263) MUTEX_GAME: === GAME START === First to 10 wins!  
I (1263) MUTEX_GAME: Human: press & hold BOOT key → lock, release → grab + unlock  
I (1263) MUTEX_GAME: AI: try lock → grab → unlock → 1s cooldown  
I (2263) MUTEX_GAME: Producer +3 → goods=3 | Human=0 AI=0  
I (3000) MUTEX_GAME: >>> Human PRESS → taking lock...  
I (3000) MUTEX_GAME: >>> Human GOT LOCK (AI blocked now!)  
I (3263) MUTEX_GAME: --- AI trying to take lock... ← AI blocks here  
I (4000) MUTEX_GAME: <<< Human RELEASE → grabbed! goods=2 Human=1 AI=0  
I (4000) MUTEX_GAME: <<< Human UNLOCKED (AI can compete now)  
I (4000) MUTEX_GAME: --- AI GOT LOCK ← AI immediately gets lock  
I (4000) MUTEX_GAME: --- AI grabbed! goods=1 Human=1 AI=1
```

6.3 Edge Cases

- **Grabbing when stock is zero:** Prints `no goods, unlock anyway`, only gives lock without grabbing
- **Human holds button continuously:** AI can never get lock, goods only increase not decrease
- **AI wins:** Solid red LED



- **Human wins:** Rainbow LED 500ms switching loop



7. FAQ

Symptom	Cause	Solution
Score inconsistency (data scrambling)	<code>g_goods</code> / <code>g_score</code> not protected by mutex	All read/write must be between <code>Take/Give</code>
AI can't get lock even when human not pressing	Other task forgot to give lock	Check all Take → Give pairs are complete
Human holds, AI blocks without error	Normal — this is mutex semantics	AI's <code>xsemaphoreTake</code> blocks until human releases lock
Deadlock	Same task takes same lock recursively	FreeRTOS mutex doesn't support recursion (use recursive mutex)
Button unresponsive	Debounce wrong / task crashed	Confirm GPIO0 configured as pull-up input
LED not lighting	<code>led_strip</code> handle NULL	Check RMT initialization